ChatGPT - Homework

L10_S2.pdf
PDF

The output must be a lecture scribe. It is exam-oriented reference material. It must be faithful to the lecture. The model must not invent content.

It should include definitions and notation, assumptions and conditions, statements of theorems or results, proofs or derivations step-by-step, worked examples with intermediate steps, and logical flow matching the lecture scribe.

Restrictions are to emphasizes discipline. It should be stated that there must be no intuition or commentary not in the lecture, no new examples, no simplifications or tutorials, and no invented content. Everything must be traceable to context.

The way to organize the scribe is it should specify the use of clear academic sections, following the lecture order, and following the structure of the example scribe if provided.

The scribe must be based only on the provided lecture material (slides, transcript, or notes). If something is unclear or missing in the lecture, it should not be filled in or guessed.

Symbols, equations, and notation must be preserved exactly as used in the lecture. Derivations must be shown line by line where they appear in the lecture.

Headings and subheadings must be used clearly. Statements of theorems, definitions, and examples must be kept clearly separated.

All intermediate steps shown in the lecture must be included. Steps that were explicitly written or explained must not be compressed.

The typical structure is:

Title

Introduction / Scope

Definitions and Notation

Results / Theorems

Proofs / Derivations

Worked Examples

Lecture Scribe

CSE400 – Fundamentals of Probability in Computing

Lecture 10: Randomized Min-Cut Algorithm

Instructor: Dhaval Patel, PhD
Date: February 5, 2026

---

1. Introduction / Scope

This lecture covers the **Min-Cut Problem**, including:

- Motivation and applications
- Definition of cut-set and minimum cut
- Successful and unsuccessful min-cut runs
- Max-Flow Min-Cut Theorem
- Deterministic Min-Cut Algorithm (Stoer–Wagner Algorithm)
- Randomized Min-Cut Algorithm (Karger's Algorithm)
- Comparison between deterministic and randomized approaches
- Theorem on probability of success of randomized min-cut

This scribe strictly follows the lecture slides. No intuition, commentary, simplifications, or additional material beyond the lecture content is included.

---

2. Min-Cut Problem

2.1 Why Use Min-Cut?

We use the min-cut algorithm in various applications to solve problems related to network connectivity, reliability, and optimization.

Applications

1. **Network Design**
   Min cut helps in improving the efficiency of communication and optimizing network flow. The algorithm is used in network design to find the minimum capacity cut.
2. **Communication Networks**
   For understanding the vulnerability of the networks to failures minimum cut can be useful. It helps building robust and fault-tolerant communication networks.
3. **VLSI Design**
   In Very Large Scale Integration (VLSI) design, the algorithm is useful for partitioning circuits into smaller components leading to reduced interconnectivity complexity.

Reference mentioned in lecture:
Section 1.5, *Application: A Randomized Min-Cut Algorithm* in *Probability and Computing (2nd Edition)*.

2.2 What is Min-Cut?

Definition: Cut-Set

A cut-set in a graph is a set of edges whose removal breaks the graph into two or more connected components.

Definition: Minimum Cut

Given a graph

$$G = (V, E)$$

with n vertices, the minimum cut — or min-cut — problem is to find a minimum cardinality cut-set in G.

Remark (From Lecture)

Min-cut algorithms, such as Karger's algorithm, are random and can be sensitive to the initial choice of edges. If the algorithm happens to contract critical edges early in the process, it might find a smaller cut.

2.3 Edge Contraction

The main operation in the algorithm is **edge contraction**, which is operation that removes an edge from a graph while simultaneously merging the two vertices.

Edge Contraction Operation

In contracting an edge (u, v):

- Merge the two vertices u and v into one vertex.
- Eliminate all edges connecting u and v.
- Retain all other edges in the graph.
- The new graph may have parallel edges but no self-loops.

---

3. Successful and Unsuccessful Min-Cut Runs

3.1 Successful Min-Cut Run

A successful min cut run refers to the success in the outcome of an algorithm designed to find the minimum cut in a graph.

(Figure: Successful Run — as shown in lecture slide)

3.2 Unsuccessful Min-Cut Run

An unsuccessful min-cut run refers to an iteration of a min-cut algorithm where the algorithm fails to correctly identify the minimum cut of a given graph.

(Figure: Unsuccessful Run — as shown in lecture slide)

---

4. Max-Flow Min-Cut Theorem

Statement of Theorem

The max-flow min-cut theorem states that:

> "In a flow network, the maximum amount of flow passing from the source to the sink is equal to the total weight of the edges in a minimum cut".

Definitions

Capacity of a Cut

Capacity of a cut = the sum of the capacity of the edges in the cut that are oriented from a vertex $\in$ X to a vertex $\in$ Y.

Minimum Cut

Minimum cut = the cut in the network that has the smallest possible capacity.

Minimum cut capacity = the capacity of the minimum cut.

Maximum Flow

Maximum flow = the largest possible flow from source S to sink T.

---

5. Deterministic Min-Cut Algorithm

5.1 Stoer–Wagner Min-Cut Algorithm

Let s and t be two vertices of a graph G.
Let $G/\{s, t\}$ be the graph obtained by merging s and t.

Then a minimum cut of G can be obtained by taking the smaller of:

- A minimum s-t-cut of G, and
- A minimum cut of $G/\{s, t\}$.

Justification (As Stated in Lecture)

The theorem holds since:

- Either there is a minimum cut of G that separates s and t, then a minimum s-t-cut of G is a minimum cut of G;
- Or there is none, then a minimum cut of $G/\{s, t\}$ does the job.

5.2 Pseudocode

Algorithm 1: MinimumCutPhase(G, a)

1. $A \leftarrow \{a\}$
2. while $A \neq V$ do
    3. add to A the most tightly connected vertex;
3. return the cut weight as the “cut of the phase”;

Algorithm 2: MinimumCut(G)

1. while $|V| \geqslant 1$ do
    2. choose any a from V;
    3. MinimumCutPhase(G, a);
    4. if the cut-of-the-phase is lighter than the current minimum cut then
    5. store the cut-of-the-phase as the current minimum cut;
    6. shrink G by merging the two vertices added last;
2. return the minimum cut;

---

6. Randomized Min-Cut Algorithm

6.1 Why Randomized Algorithm?

Randomized algorithms provide a probabilistic guarantee of success. It provides a more accurate estimate of the minimum cut with fewer iterations.

Additional points listed in lecture:

- Efficiency
- Parallelization
- Approximation Guarantees
- Avoidance of Worst-Case Instances
- Heuristic Nature
- Robustness

6.2 Karger’s Randomized Algorithm

(Title slide introducing Karger's algorithm; details correspond to randomized contraction method described earlier.)

6.3 Comparison: Deterministic vs Randomized Min-Cut

Any specific problem is responsible for deciding the approach.

Deterministic Min Cut

- It always guarantees an exact minimum cut.
- It may have higher time complexity for large graphs.
- The Stoer-Wagner algorithm has time complexity:

$$O(V \cdot E + V^2 \log V)$$

where V is the number of vertices and E is the number of edges.

Randomized Min Cut

- It provides an approximate minimum cut with high probability.
- Karger's algorithm has time complexity:

$$O(V^2)$$

where V is the number of vertices in the graph.

---

7. Theorem for Min-Cut Set

The algorithm outputs a min-cut set with probability at least $\frac{2}{n(n-1)}$.

---

8. Python Simulation

Instruction from lecture:

- Open Campuswire post regarding today's lecture (L10).
- Download the `.ipynb` file pasted in the comments.

End of Lecture 10

This scribe strictly follows the lecture slides for Lecture 10: Randomized Min-Cut Algorithm.
No additional explanations, intuition, examples, or derivations beyond the lecture content have been included.